[Developer] Google Chat EDTS API Event Payload Documentation

API Endpoints

/create_msg

Endpoint for creating and sending messages to Google Chat.

Flowchart:

Request

- Method: POST
- Content-Type: application/json

Message Types (msg_type)

1. leave_approval: Used for leave-related notifications
2. link_preview: Used for messages containing links and previews
3. upload_image: Used for messages containing image
4. send_attachment : Used for messages containing document links

Leave Approval

Creates a new message in the system for leave approval notifications.

Request

```JSON
{
  "message": {
    "data": "<base64_encoded_string>"
  }
}
```

The base64_encoded_string contains the following JSON structure:

```JSON
{
  "project_id": string,    // Google Cloud project ID
  "employee_id": string,   // Employee identifier
  "msg_type": string,      // Type of message (e.g., "leave_approval")
  "start_leave_dt": string, // Leave start date (YYYY-MM-DD format)
  "end_leave_dt": string   // Leave end date (YYYY-MM-DD format)
}
```

Example Request

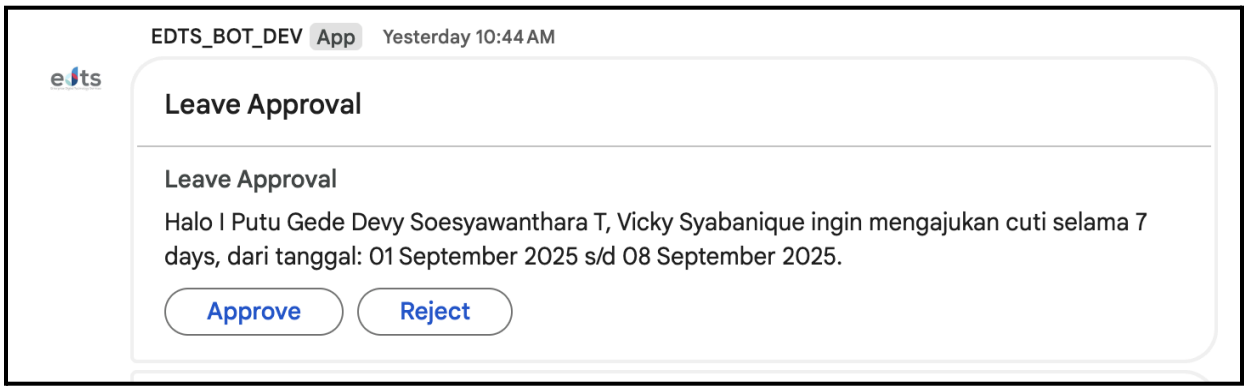
```JSON
curl -X POST 'http://localhost:5000/create_msg' \
-H 'Content-Type: application/json' \
-d '{
  "message": {
    "data": "<base64_encoded_payload>"
  }
}'
```

Sample Payload Data (Pre-encoding)

JSON

```
{
  "project_id": "idf-corp-dev",
  "employee_id": "EDTS90206",
  "msg_type": "leave_approval",
  "start_leave_dt": "2025-09-01",
  "end_leave_dt": "2025-09-08"
}
```

Sample Output



Gambar 1.1 Leave Approval

Link Preview

Payload Structure

JSON

```
{
  "message": {
    "data": "base64_encoded_string"
  }
}
```

Example Request

```JSON
curl -X POST 'http://localhost:5000/create_msg' \
-H 'Content-Type: application/json' \
-d '{
  "message": {
    "data": "<base64_encoded_payload>"
  }
}'
```

Sample Payload Data (Pre-encoding)

The base64_encoded_string contains a JSON object with the following structure:

```JSON
{
  "project_id": string,      // Project identifier
(e.g.,"idf-corp-dev")
  "employee_id": string,     // Employee identifier (e.g.,
"EDTS90206")
  "msg_type": string,        // Message type identifier
"header": {
     "title": "Contoh Attachement",
     "subtitle": "Contoh Attachment file"
}, // Optional: this object is required when you want to send
report (especially for /link_preview type)
  "widgets": array          // Optional: Array of widget objects
for rich message content
}
```

Widgets Structure

When using rich content (especially for link_preview type), you can include widgets:

JSON

```
"widgets": [

  {

    "text": string       // Text content with HTML formatting
support

  },

  {

    "image": {

      "image_url": string,      // URL of the image

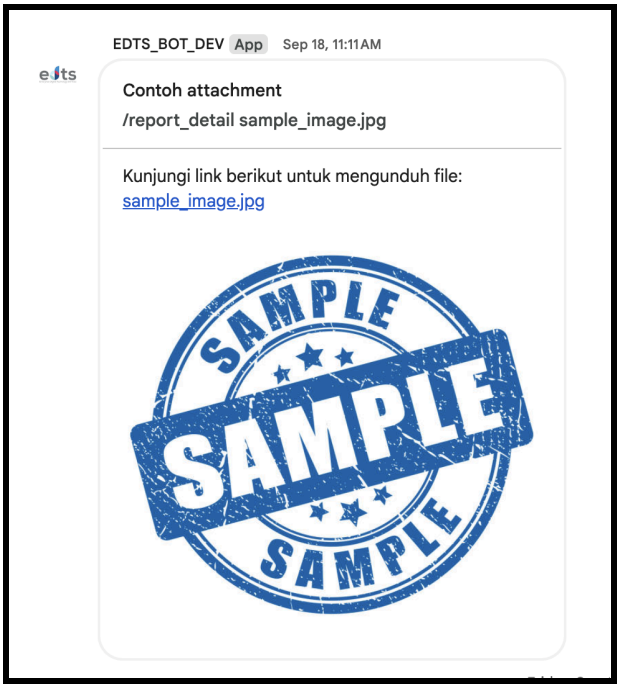
      "open_image_link": string, // URL to open when image is
clicked

      "alt_text": string        // Alternative text for
accessibility

    }

  }
```

Sample Output



Gambar 1.2 Link Preview

Upload Image

Example Request

```
JSON
curl -X POST 'http://localhost:5000/create_msg' \
-H 'Content-Type: application/json' \
-d '{
  "message": {
    "data": "<base64_encoded_payload>"
  }
}'
```

Sample Payload Data (Pre-encoding)

The base64_encoded_string contains a JSON object with the following structure:

```JSON
{
  "project_id": str, // Project identifier (e.g.,"idf-corp-dev")
  "employee_id": str,// Employee identifier (e.g., "EDTS90206")
  "msg_type": "upload_img",
  "doc_link": "https://lh3.googleusercontent.com/d/[image_link]",

}
```

Sample Output



Gambar 1.3 Upload Image

Send Attachments

Example Request

JSON

```
curl -X POST 'http://localhost:5000/create_msg' \
-H 'Content-Type: application/json' \
-d '{
  "message": {
    "data": "<base64_encoded_payload>"
  }
}'
```

Sample Payload Data with Employee ID as part of the payload

The base64_encoded_string contains a JSON object with the following structure:

JSON

```
{
    "project_id" : str,// Project identifier
(e.g.,"idf-corp-dev")
    "employee_id" : str,// Employee identifier (e.g.,
"EDTS90206")
    "msg_type" : "send_attachments_with_employee_id",
    "details" : [...]
}
```

Sample Payload Data with Employee ID as part of the payload

The base64_encoded_string contains a JSON object with the following structure:

JSON

```
{
```

```
    "project_id": str,// Project identifier
(e.g.,"dev-idm-sd7-chatbot")
    "msg_type" : "send_attachments",
    "space_id" : str,// Space identifier (e.g., "A51S31098")
    "rpt_period_dt" : str //Report date release
(e.g.,"2025-08-01"),
    "details" : [...]
}
```

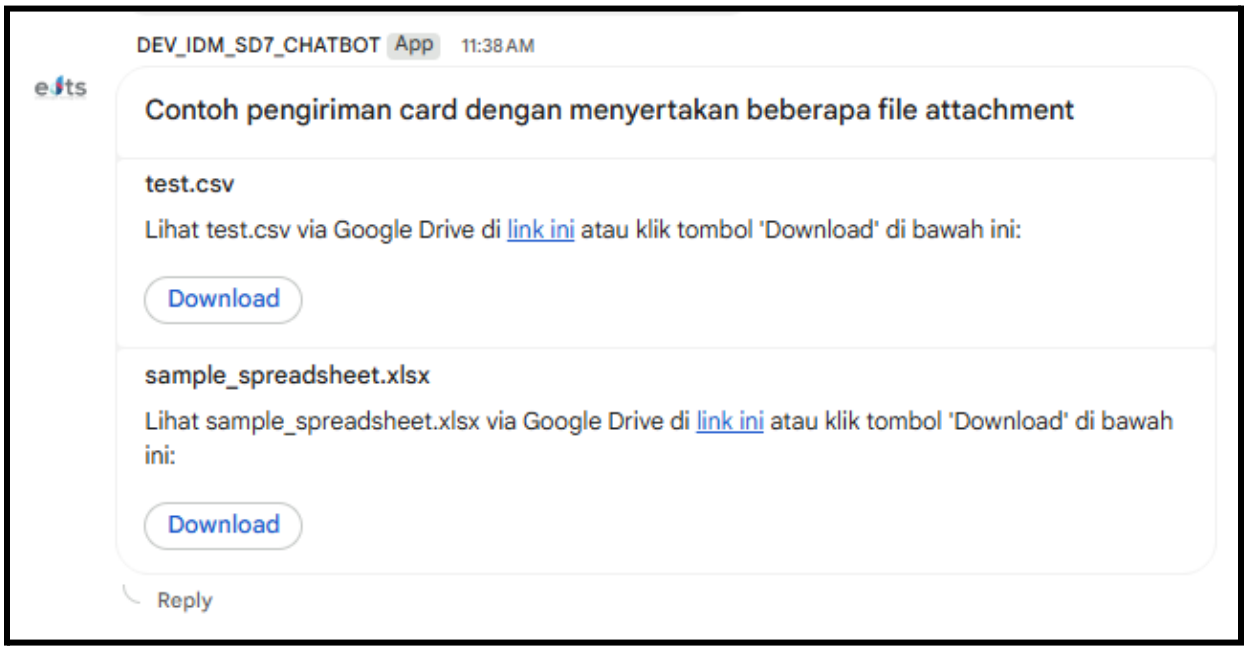
Detail structure:
Can be repeated as many times as needed.

JSON

```
"details" : [
    {
        "google_drive_path_url" : str,// URL for google drive
folder
(e.g.,"https://drive.google.com/drive/folders/[folder1_id]"),
        "gdrive_path":  str,// URL for google drive file
(e.g.,"https://drive.google.com/file/d/[file_id]/"),
        "gdrive_filename":  str,// name of the file (with
extention) (e.g. "test.csv"),
     }
]
```

- Google_drive_path_url → URL for google drive folder path
- Gdrive_path → URL for google drive file
- Gdrive_filename → name of the file (with extention)

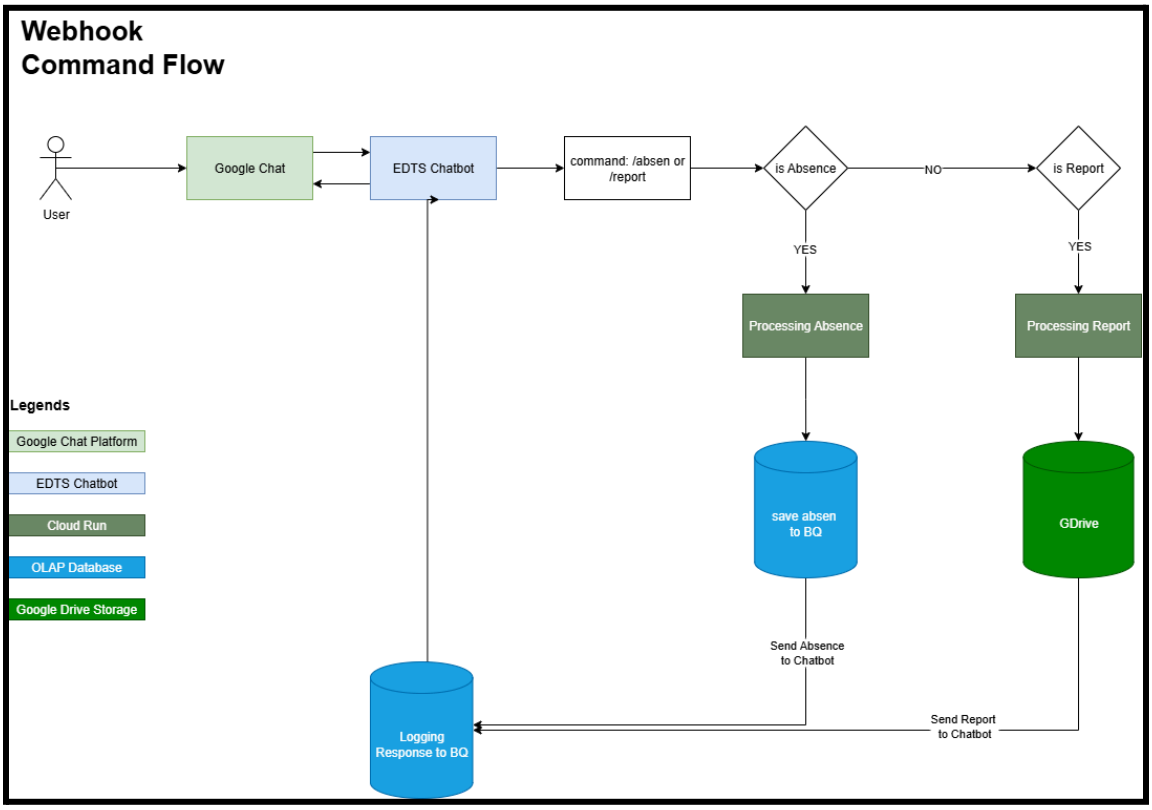
Sample Output



Gambar 1.3 Upload Image

[Developer] Google Chat Webhook Event Payload Documentation

Webhook Flow



Gambar 1.1 Webhook Flow

Triggering Commands via Webhook

When a bot receives an event from Google Chat, Google sends an **HTTP POST request** to our webhook endpoint. The body is a **JSON payload**.

The general payload structure looks like:

```
JSON

{
  "commonEventObject": { ... },
  "authorizationEventObject": { ... },
  "chat": { ... }
}
```

1. commonEventObject

General information about the event environment.

Field	Type	Description
userLocale	String	The user's locale (e.g.,"en","id").
hostApp	String	Always"CHAT".
platform	String	The platform where the event happened ("WEB","MOBILE","IOS","ANDROID", etc.).

2. authorizationEventObject

Authorization and identity information for the bot.

Field	Type	Description
systemIdToken	String	A JWT (ID token) that can be used to verify the request.

3. chat

The main payload of the event. This varies depending on the event type.

3.1. Message Event

Triggered when a user sends a message (plain text, mention, or reply in a thread).

[payload_example]

```json
"chat": {
  "user": { ... },
  "eventTime": "2025-09-18T02:22:14.294Z",
  "messagePayload": {
    "space": { ... },
    "message": { ... }
  }
```

```
}
```

- **user** → user who triggered the event.
- **messagePayload**
  - space → information about the space (DM, group, etc.).
  - message → details of the message (text, sender, thread, annotations, etc.).

3.2. Added to space Event

Triggered when the bot is added to or removed from a space.

[payload example]

Added to space

```JSON
"chat": {
  "user": { ... },
  "eventTime": "2025-09-18T02:25:10.111Z",
  "addedToSpacePayload": {
    "space": {
       "name": "spaces/[space_id]",
        "type": "[type]",
        "displayName": "Test",
        "spaceThreadingState": "THREADED_MESSAGES",
        "spaceType": "[space type]",
        "spaceHistoryState": "HISTORY_ON",
        "lastActiveTime": "[last_active_time]",
        "membershipCount": {...},
        "customer": "customers/C038ddaw0",
        "spaceUri": "https://chat.google.com/room/[space_id]"}
  }
}
```

- **space** → details of the space.

3.3. Removed to space

JSON

```
"chat": {
  "user": { ... },
  "eventTime": "2025-09-18T02:25:10.111Z",
  "removedFromSpacePayload": {
    "space": { ... }
  }
}
```

- **space** → details of the space.

3.4. Command Event

[payload_example]

3.4.1. Command Event with type: Slash Command

JSON

```
"chat": {
  "user": { ... },
  "eventTime": "2025-09-18T02:25:10.111Z",
  "appCommandPayload": {
      "appCommandMetadata": {
        "appCommandId": [command_id],
        "appCommandType": "SLASH_COMMAND"
      }
    "space": { ... },
    "message": { ... },
    "configCompleteRedirectUri": "..."
  }
}
```

3.4.2. Command Event with type: Quick Command

```JSON
"chat": {
  "user": { ... },
  "eventTime": "2025-09-18T02:25:10.111Z",
  "appCommandPayload": {
      "appCommandMetadata": {
        "appCommandId": [command_id],
        "appCommandType": "QUICK_COMMAND"
      }
    "space": { ... },
    "message": { },
    "configCompleteRedirectUri": "..."
  }
}
```

- **space** → details of the space.
- **message** → details of the message (text, sender, thread, annotations, etc.).

4. Important Objects

4.1. User Object

```JSON
   "user": {
      "name": "users/[user_id]",
      "displayName": "[user_display_name]",
      "avatarUrl":
"https://lh3.googleusercontent.com/a/[display_url",
      "email": "[user_email]",
      "type": "HUMAN",
      "domainId": "[domain_id]"
    },
```

4.2. Space Object

4.2.1. Group Space

```json
"space": {
      "name": "spaces/[space_id]",
      "type": "ROOM",
      "displayName": "Test",
      "spaceThreadingState": "THREADED_MESSAGES",
      "spaceType": "SPACE",
      "spaceHistoryState": "HISTORY_ON",
      "lastActiveTime": "2026-02-25T09:13:40.804575Z",
      "membershipCount": {
           "joinedDirectHumanUserCount": 1
      },
      "customer": "customers/[customer_id]",
      "spaceUri": "https://chat.google.com/room/[space_id]"
}
```

4.2.2. Direct message

```json
"space": {
      "name": "spaces/[space_id]",
      "type": "DM",
      "singleUserBotDm": true,
      "spaceThreadingState": "THREADED_MESSAGES",
      "spaceType": "DIRECT_MESSAGE",
      "spaceHistoryState": "HISTORY_ON",
      "lastActiveTime": "2026-03-02T01:56:43.298591Z",
      "membershipCount": {
           "joinedDirectHumanUserCount": 1
      },
      "spaceUri": "https://chat.google.com/dm/[space_id]"
},
```

4.3. Message Object

JSON

```
"message": {
  "name": "spaces/[space_id]/messages/[message_id]",
  "sender": {...},
  "createTime": "2026-03-02T01:56:43.298591Z",
  "text": "@[bot_name] /daftar IDM[id_karyawan] -
[nama_karyawan]",
  "annotations": [{...}],
  "thread": {...}
  },
  "space": {...},
  "arguzmentText": " /daftar IDM[id_karyawan] - [nama_karyawan]",
  "retentionSettings": {
    "state": "PERMANENT"
  },
  "messageHistoryState": "HISTORY_ON",
  "formattedText": "@[bot_name] /daftar IDM[id_karyawan] -
[nama_karyawan]"
}
```

- **space** → details of the space.
- **Annotation** → is type of interaction, eg: user_mention.
- **User inside annotation** → indicates which user is mentioned.

None

```
"annotations": [
  {
    "type": "USER_MENTION",
    "startIndex": 0,
    "length": 20,
    "userMention": {
      "user": {
        "name": "users/[bot_id]",
        "displayName": "[bot_name]",
```

```
        "avatarUrl":
"https://lh6.googleusercontent.com/proxy/<>",
        "type": "BOT"
      },
      "type": "MENTION"
    }
  }
]

"thread": {
   "name": "spaces/[space_id]/threads/5vTy3yIoIJc",
   "retentionSettings": {
   "state": "PERMANENT"
}
```

- thread → indicated thread the message is involved in

None

```
"thread": {
   "name": "spaces/[space_id]/threads/[message_id]",
   "retentionSettings": {
   "state": "PERMANENT"
}
```

5. Summary of Possible Events

Event	Payload field	When it's triggered
Message	messagePayload	A user sends a message, mentions the bot, or replies in a thread
Added to Space	addedToSpacePayload	The bot is added to a space
Removed from Space	removedFromSpacePayload	The bot is removed from a space

Command Payload	appCommandPayload	A user selects the app command from the suggestion list

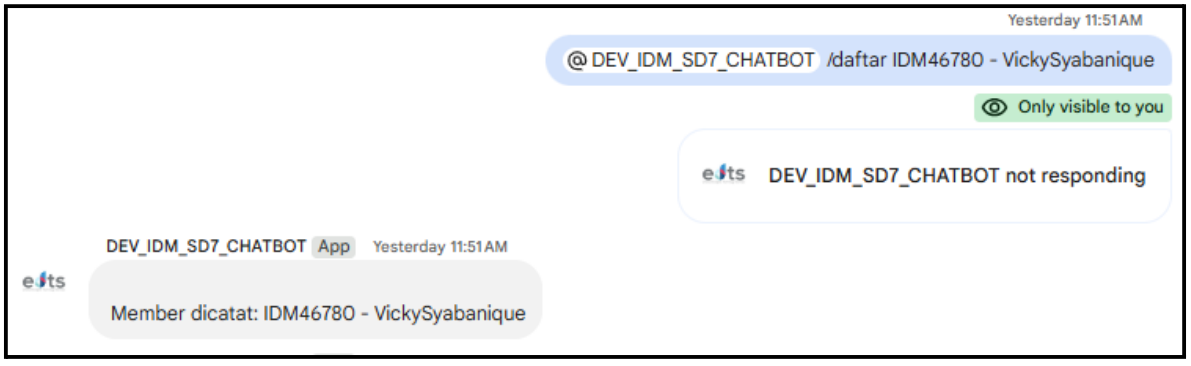
[User] Using the Google Chat Bot: A User Guide

This guide provides a comprehensive overview of how to interact with the Google Chat bot, covering various methods of communication and understanding its responses.

1. Executing Commands

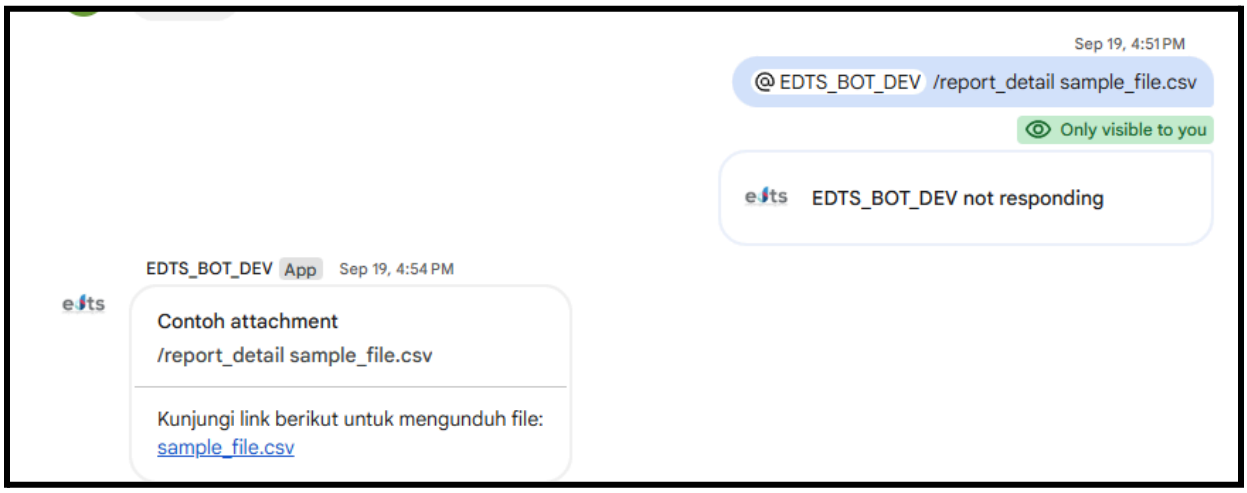
Commands are special instructions that begin with a forward slash (/). These trigger specific functions or retrieve information from the bot.

- **/daftar IDM[ID Number] - [Full Name]**
  - **Purpose:** To record your attendance or absence with necessary details.
  - **Details:**
    - [ID Number]: Your unique identification number.
    - [Full Name]: Your complete name.
  - **Example:** /daftar - IDM12345 Levi Ackerman
  - **Output :**



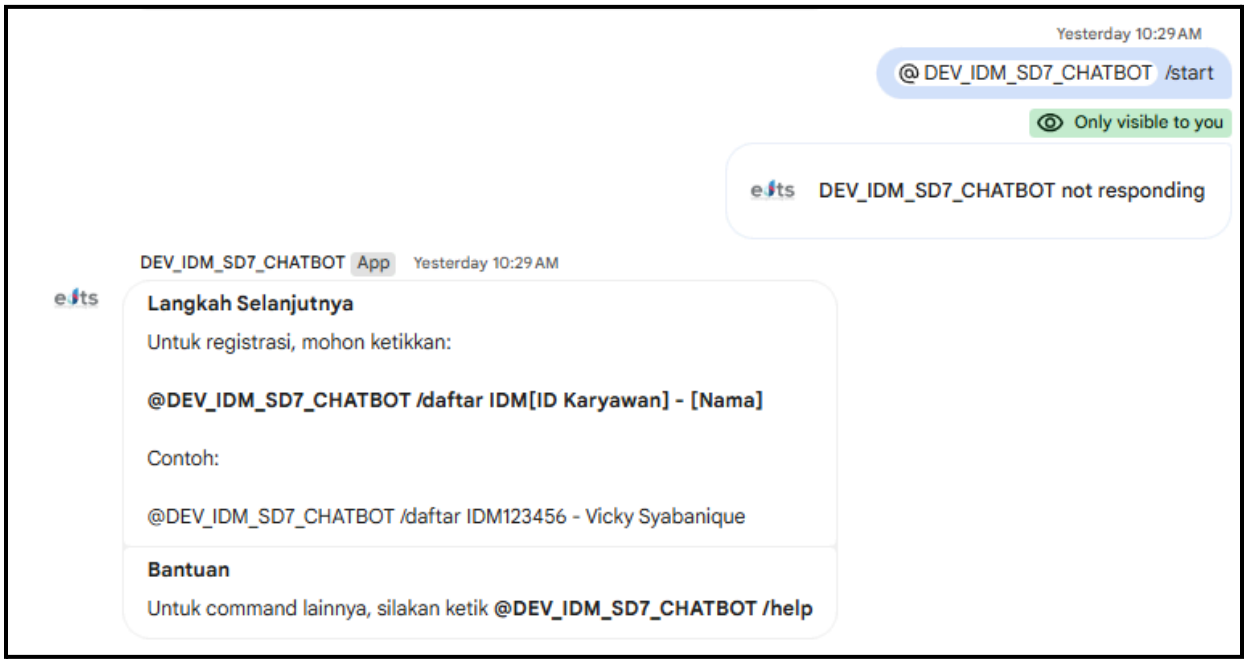
Gambar 1.1 Output command daftar

- **/report_detail [filename]**
    - **Purpose:** To request a specific report, often linked to a particular file.
    - **Details:**
        - [filename]: The name of the file you wish to get a report for (e.g., quarterly_report.pdf).
    - **Example:** /report_detail sales_data_Q2.xlsx
    - **Output :**



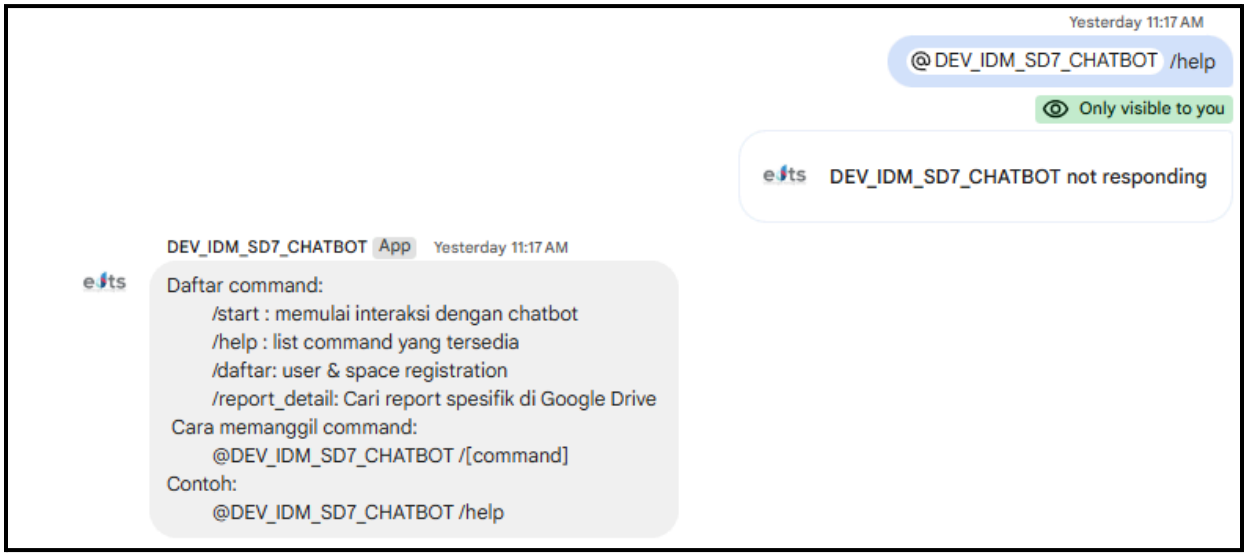
Gambar 1.2 Output command report_detail

- **/start**
    - **Purpose:** onboarding entry point. It introduces the next step needed to be taken by the user and guides users on how to interact with it.
    - **Example:** /start
    - **Output :**



Gambar 1.3 Output command start

- **/help**
  - **Purpose:** provides contextual assistance by listing available commands and explaining how to use them.
  - **Example:** /help
  - **Output :**



Gambar 1.3 Output command help

3. Replying within Threads

The bot is designed to understand and respond within the context of Google Chat threads. You can reply directly to any message in a thread, and the bot will process your input as part of that ongoing conversation.

- **How to:** When a message is part of a thread, either click "Reply in thread" or type your message directly into the reply box that appears within the thread view.
- **Example:** If the bot asks a follow-up question in a thread, you can provide your answer as a reply within that thread.

4. Understanding Bot Notifications and Interactions

The bot will proactively inform you about certain events and respond to interactive elements.

- **When Added to a Space:** The bot will send a welcome message when it is successfully added to a Google Chat space.
- **When Removed from a Space:** The bot may send a departure message if it is removed from a space.
- **Card Clicks:** If the bot sends interactive cards (e.g., with buttons or forms), clicking on these elements will trigger specific actions or responses from the bot, such as approving requests or submitting data.